

## 9. Processos Gaussianos

### 9.1 Revisitando a priori Gaussiana

Como discutimos anteriormente, prioris gaussianas são extremamente populares em modelos Bayesianos para regressão. Uma escolha comum, por exemplo, é colocar uma priori isotônica  $\mathcal{N}(\mathbf{0}, c\mathbf{I})$  sobre o vetor de pesos  $\boldsymbol{\theta}$ . Nesse caso, a priori sobre  $\boldsymbol{\theta}$  também induz implicitamente uma priori sobre  $f(\mathbf{X}') = \mathbf{X}'\boldsymbol{\theta}$  para qualquer  $\mathbf{X}' \in \mathbb{R}^{N' \times d+1}$  e  $N' \in \mathbb{N}^+$ . Mais especificamente, como  $f(\mathbf{X}')$  é uma transformação linear de variáveis Gaussianas, essa priori é Gaussiana com vetor de médias  $\boldsymbol{\mu}(\mathbf{X}')$  e matriz de covariância  $\Sigma(\mathbf{X}', \mathbf{X}')$  dados por:

$$\begin{aligned}\boldsymbol{\mu}(\mathbf{X}') &= \mathbb{E}_{\boldsymbol{\theta}}[\mathbf{X}'\boldsymbol{\theta}] = \mathbf{X}'\mathbb{E}_{\boldsymbol{\theta}}[\boldsymbol{\theta}] = \mathbf{0} \\ \Sigma(\mathbf{X}', \mathbf{X}') &= \mathbb{E}_{\boldsymbol{\theta}}[(\mathbf{X}'\boldsymbol{\theta} - \mathbf{0})(\mathbf{X}'\boldsymbol{\theta} - \mathbf{0})^\top] = c\mathbf{X}'\mathbf{X}'^\top\end{aligned}$$

Com essas observações em mente, podemos abstrair  $\boldsymbol{\theta}$  totalmente do nosso processo de aprendizado usando a seguinte priori sobre os valores de  $f$ :

$$f(\mathbf{X}') \sim \mathcal{N}(\mathbf{0}, \Sigma(\mathbf{X}', \mathbf{X}')) \quad \forall \mathbf{X}' \in \mathbb{R}^{N' \times d+1}, N' \in \mathbb{N}^+,$$

que é uma instância específica de um processo estocástico conhecido como processo Gaussian (*Gaussian process*, GP). De forma geral,  $(f(\mathbf{x}))_{\mathbf{x} \in \mathcal{X}}$  define um processo Gaussiano se qualquer vetor  $[f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)]^\top$  com  $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathcal{X}$  segue uma distribuição normal multivariada.

É importante ressaltar que a matriz de covariância  $\Sigma(\mathbf{X}', \mathbf{X}')$  é proporcional à matriz Gramiana  $\mathbf{K}$  (de produtos internos) dos vetores linha de  $\mathbf{X}'$ , i.e.,  $K_{i,j} = \mathbf{x}'_i \cdot \mathbf{x}'_j$  onde  $\mathbf{x}'_i$  e  $\mathbf{x}'_j$  denotam os vetores nas linhas  $i$  e  $j$  de  $\mathbf{X}'$ , respectivamente. Além disso, incorporar uma função de expansão de base  $\phi$  no modelo da Equação 9.1 apenas implica em redefinir as entradas de  $\mathbf{K}$  como  $K_{i,j} = k(\mathbf{x}'_i, \mathbf{x}'_j) = \phi(\mathbf{x}'_i) \cdot \phi(\mathbf{x}'_j)$ . Em outras palavras, nosso GP sobre  $f$  pode ser completamente caracterizado por uma função de produto interno generalizada  $k$  — também conhecida como função de kernel. Por simplicidade notacional, denotaremos que  $f$  segue uma priori de GP como  $f \sim \text{GP}(\mathbf{0}, k)$ . Nesse capítulo, assumiremos que a média de  $f$  é zero a priori, no entanto, seria possível utilizar uma função de média arbitrária  $\mu(\cdot)$  com poucas alterações nos nossos desenvolvimentos.

Do ponto de vista de interpretação, funções de kernel nos permitem diretamente expressar como regularidades no espaço de entrada devem ser refletidos no espaço de saída. Além disso, existem casos em que  $\phi$  é computacionalmente intratável, mas seu kernel correspondente tem forma simples. Por exemplo, o kernel exponencial quadrático (ou Gaussiano), dado por  $k(\mathbf{x}, \mathbf{x}') = \exp\{-\frac{\|\mathbf{x} - \mathbf{x}'\|_2^2}{2\gamma^2}\}$  é gerado a partir de uma expansão de base "infinita" — veja a Seção 4.2 do livro texto por Rasmussen e Williams (2006) para mais detalhes.

**Funções de kernel comuns.** Na literatura de GPs, existe uma variedade de funções de kernel criadas para modelar fenômenos distintos. No entanto, alguns kernels são extremamente populares, como:

1. Exponencial quadrático (ou Gaussiano):

$$k(\mathbf{x}, \mathbf{x}') = e^{-\frac{\|\mathbf{x} - \mathbf{x}'\|_2^2}{2\gamma^2}},$$

onde  $\gamma^2$  define é um hiperparâmetro que controla a suavidade da função de kernel;

2. Racional quadrático:

$$k(\mathbf{x}, \mathbf{x}') = \sigma^2 \left( 1 + \frac{\|\mathbf{x} - \mathbf{x}'\|_2^2}{2\alpha\gamma^2} \right)^{-\alpha},$$

que corresponde à uma soma pon

3. Periódico:

$$k(\mathbf{x}, \mathbf{x}') = e^{-2\gamma^{-2} \sin^2(p^{-1}\pi\|\mathbf{x} - \mathbf{x}'\|_2)},$$

que tem natureza periódica, com período  $p$ .

**Construindo funções de kernel.** Funções a família das funções de kernel é fechada por um número de operações. Isto é, é possível manipular um kernel  $k$  de várias maneiras diferentes e ainda assim obter um kernel válido. Isso é extremamente útil em cenários em que precisamos, e.g., combinar propriedades de diferentes funções de kernel para refletir um fenômeno. Por exemplo, as seguintes operações resultam em kernels válidos:

1. Multiplicação por constante  $c > 0$ :  $k'(\mathbf{x}, \mathbf{x}') = ck(\mathbf{x}, \mathbf{x}')$ ;
2. Produto:  $k'(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}')k_2(\mathbf{x}, \mathbf{x}')$ ;
3. Soma:  $k'(\mathbf{x}, \mathbf{x}') = k_1(\mathbf{x}, \mathbf{x}') + k_2(\mathbf{x}, \mathbf{x}')$ ;
4. Exponenciação:  $k'(\mathbf{x}, \mathbf{x}') = e^{k_1(\mathbf{x}, \mathbf{x}')}$ ;
5. Multiplicação por função escalar avaliada em  $\mathbf{x}$  e  $\mathbf{x}'$ :  $k'(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})f(\mathbf{x}')k(\mathbf{x}, \mathbf{x}')$ .

## 9.2 GPs para regressão

Em problemas de regressão, é comum que a variável de resposta  $y_n$  para a entrada  $\mathbf{x}_n$  seja observada com algum ruído. Seguindo o Capítulo 4, suponha que  $y = g(\mathbf{x}) = f(\mathbf{x}) + \epsilon$  com  $\epsilon \sim \mathcal{N}(0, \sigma^2)$ . Note que isso é equivalente a dizer que  $y|\mathbf{x} \sim \mathcal{N}(f(\mathbf{x}), \sigma^2)$ , portanto, podemos descrever um modelo qualquer de regressão com priori GP como:

$$\begin{aligned} y|f, \mathbf{x} &\sim \mathcal{N}(f(\mathbf{x}), \sigma^2) \\ f &\sim \text{GP}(\mathbf{0}, k) \end{aligned} \quad (9.1)$$

Como é de costume, estamos interessados em usar o modelo acima a o conjunto de treino  $\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N$  para fazer predições para, e.g., a variável de resposta  $y_*$  relativa à um novo vetor de atributos  $\mathbf{x}_*$ . Em outras palavras, queremos avaliar  $p(y_*|\mathbf{x}_*, \mathbf{x}_1, y_1, \dots, \mathbf{x}_N, y_N)$ . No entanto, existe um detalhe no modelo acima que facilitará nossa vida: a nossa combinação de priori e verossimilhança induz um processo Gaussiano sobre  $g$ . Mais especificamente, para qualquer  $\mathbf{X}' \in \mathbb{R}^{M \times d}$ ,  $g(\mathbf{X}')$  pode ser descrito como uma soma de duas variáveis aleatórias Gaussianas:  $f(\mathbf{X}')$  e um vetor  $M$ -dimensional  $\epsilon$  com distribuição  $\mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$ . Portanto, temos que  $g(\mathbf{X}')$  segue  $\mathcal{N}(\mathbf{0}, k(\mathbf{X}', \mathbf{X}') + \sigma^2 \mathbf{I})$ . Tomando  $\mathbf{X}' = [\mathbf{x}_1, \dots, \mathbf{x}_N, \mathbf{x}_*]^\top$  temos:

$$\begin{bmatrix} y_* \\ y_1 \\ \vdots \\ y_N \end{bmatrix} \sim \mathcal{N} \left( \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \begin{bmatrix} k(\mathbf{x}_*, \mathbf{x}_*) + \sigma^2 & k(\mathbf{x}_*, \mathbf{x}_1) & \dots & k(\mathbf{x}_*, \mathbf{x}_N) \\ k(\mathbf{x}_1, \mathbf{x}_*) & k(\mathbf{x}_1, \mathbf{x}_1) + \sigma^2 & \dots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_*) & k(\mathbf{x}_N, \mathbf{x}_1) & \dots & k(\mathbf{x}_N, \mathbf{x}_N) + \sigma^2 \end{bmatrix} \right),$$

onde o lado direito está implicitamente condicionado nas entradas  $\mathbf{x}_1, \dots, \mathbf{x}_N$  e  $\mathbf{x}_*$ . Portanto, podemos obter a distribuição  $p(y_* | \mathbf{x}_*, \mathbf{x}_1, y_1, \dots, \mathbf{x}_N, y_N)$  simplesmente condicionando a Gaussiana acima nos valores observados  $y_1, \dots, y_N$ , i.e.,  $p(y_* | \mathbf{x}_*, \mathbf{x}_1, y_1, \dots, \mathbf{x}_N, y_N) = \mathcal{N}(\boldsymbol{\mu}_*, \mathbf{S}_*)$  com parâmetros  $\boldsymbol{\mu}_*, \mathbf{S}_*$  dados por

$$\begin{aligned} \boldsymbol{\mu}_* &= k(\mathbf{x}_*, \mathbf{X}) (k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I})^{-1} \mathbf{y} \\ \mathbf{S}_* &= k(\mathbf{x}_*, \mathbf{x}_*) + \sigma^2 - k(\mathbf{x}_*, \mathbf{X}) (k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I})^{-1} (k(\mathbf{x}_*, \mathbf{X}))^\top \end{aligned}$$

onde  $\mathbf{X}$  é a matriz com dados de treinamento e  $\mathbf{y}$  é o vetor de suas respectivas respostas. Uma observação interessante é que  $\boldsymbol{\mu}_*$  é, essencialmente, uma combinação linear das entradas em  $\mathbf{y}$ . Finalmente, vale ressaltar que  $\sigma^2$  é comumente tratado como um hiperparâmetro, que podemos escolher com auxílio de um conjunto de validação.

### 9.2.1 Escolhendo os parâmetros do kernel

Já sabemos como construir um GP simples para regressão. No entanto, não discutimos como escolher os parâmetros da noção função de kernel (e.g., a largura de banda  $\gamma$  do kernel Gaussiano). Como fazer isso? Na literatura de GPs, a saída comum é maximizar a esperança da verossimilhança sob a nossa priori, i.e.,  $p(y_1, \dots, y_n | \mathbf{x}_1, \dots, \mathbf{x}_N) = \mathbb{E}_{f \sim \text{GP}(\mathbf{0}, k)}[\mathcal{N}(\mathbf{y} | f(\mathbf{X}), \sigma^2 \mathbf{I})]$ . Mais concretamente, denotando de maneira genérica os parâmetros da função de kernel por  $\boldsymbol{\omega}$ , os parâmetros ótimos podem ser calculados como:

$$\begin{aligned} \hat{\boldsymbol{\omega}} &= \arg \max_{\boldsymbol{\omega} \in \Omega} \log p(y_1, \dots, y_n | \mathbf{x}_1, \dots, \mathbf{x}_N) \\ &= \arg \max_{\boldsymbol{\omega} \in \Omega} \log \mathcal{N}(\mathbf{y} | \mathbf{0}, k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}) \\ &= \arg \max_{\boldsymbol{\omega} \in \Omega} -\frac{1}{2} \log \det(k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}) - \frac{1}{2} \mathbf{y}^\top (k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I})^{-1} \mathbf{y} \end{aligned}$$

O procedimento descrito acima é comumente conhecido como *type II maximum likelihood*, maximização da verossimilhança marginal e maximização da evidência (note que estamos maximizando o denominador da regra de Bayes). Vale ressaltar que, ao contrário de MLE para regressão linear e logística, o problema acima costuma não ser convexo em  $\boldsymbol{\omega}$ . Por isso, é ideal conduzir otimização *multi-start* — e.g., rodando SGD usando diferentes pontos iniciais e escolhendo o melhor ótimo local obtido.

Note também que o procedimento que descrevemos assume que o ruído observacional  $\sigma^2$  é uma constante. Nesse caso, poderíamos estimar os parâmetros do kernel para cada valor de  $\sigma^2$  usando o conjunto de treino e escolher o  $\sigma^2$  que resulta na maior evidência. Outra opção é otimizar  $\sigma^2$  juntamente com  $\boldsymbol{\omega}$ , i.e.:

$$\hat{\sigma}^2, \hat{\boldsymbol{\omega}} = \arg \max_{\boldsymbol{\omega} \in \Omega, \sigma^2 \in \mathbb{R}} -\frac{1}{2} \log \det(k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I}) - \frac{1}{2} \mathbf{y}^\top (k(\mathbf{X}, \mathbf{X}) + \sigma^2 \mathbf{I})^{-1} \mathbf{y}.$$

No entanto, otimizar  $\sigma^2$  e  $\omega$  juntos pode tornar nosso processo de aprendizado instável. Para aliviar esse problema, a comunidade de GPs em ML costuma usar algumas heurísticas de inicialização. A heurística mais comum consiste em fixar (a priori) a razão  $r^2$  entre a variância do verossimilhança e da priori. Assumindo que nosso kernel é da forma  $k = c^2 k'$ , o primeiro passo é inicializar  $c$  com a variância das saídas de treinamento  $y_1, \dots, y_N$ . Subsequentemente, inicializamos o ruído observacional  $\sigma^2$  com  $c^2/r^2$ . Nesse contexto, repetimos o processo de otimização para diferentes valores de  $r^2$  e escolhemos o resultado que leva ao melhor ótimo local.

## Bibliografia

- Ahn, K., Yun, C., & Sra, S. (2020). SGD with shuffling: optimal rates without component convexity and large epoch requirements. *Advances in Neural Information Processing Systems (NeurIPS)* (ver página 11).
- Atkeson, C. G., Moore, A. W., & Schaal, S. (1997). Locally Weighted Learning. *Artificial Intelligence Review*, 11(1–5) (ver página 20).
- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press. (Ver página 22).
- Broomhead, D., & Lowe, D. (1988). Multivariable Functional Interpolation and Adaptive Networks. *Complex Systems*, 2, 321–355 (ver página 30).
- Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27 (ver página 19).
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems (MCSS)*, 2(4), 303–314 (ver página 59).
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14, 179–211 (ver página 64).
- Fukushima, K. (1975). Cognitron: A Self-Organizing Multilayer Neural Network. *Biological Cybernetics*, 20, 121–136 (ver página 59).
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366 (ver página 59).
- Lloyd, S. P. (1982). Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28, 129–137 (ver página 30).
- Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press. (Ver página 22).
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115–133 (ver páginas 5, 56).
- Micchelli, C. A. (1986). Interpolation of scattered data: Distance matrices and conditionally positive definite functions. *Constr. Approx.*, 2, 11–22 (ver página 30).
- Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press. (Ver página 57).
- Mohamed, S., Rosca, M., Figurnov, M., & Mnih, A. (2020). Monte Carlo Gradient Estimation in Machine Learning. *Journal of Machine Learning Research (JMLR)* (ver página 42).
- Nair, V., & Hinton, G. (2010). Rectified linear units improve restricted boltzmann machines. *International conference on machine learning (ICML)* (ver página 59).
- Nguyen, L., Tran-Dinh, Q., Phan, D., Nguyen, P., & van Djik, M. (2021). A Unified Convergence Analysis for Shuffling-Type Gradient Methods. *Journal of machine learning research (JMLR)* (ver página 11).
- Novikoff, A. B. (1962). On Convergence Proofs on Perceptrons. *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615–622 (ver página 57).

- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian processes for machine learning*. MIT Press. (Ver página 52).
- Rosenblatt, F. (1957). *The perceptron: A perceiving and recognizing automaton (Project PARA)* (relatório técnico). Cornell University (Technical Report 85-460-1). (Ver página 56).
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *nature*, 323(6088), 533–536 (ver páginas 57, 59).
- Safran, I., & Shamir, O. (2021). Random Shuffling Beats SGD Only After Many Epochs on Ill-Conditioned Problems. *Advances in Neural Information Processing Systems (NeurIPS)* (ver página 11).
- Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning - From Theory to Algorithms*. Cambridge University Press. (Ver página 44).
- Telgarsky, M. (2016). benefits of depth in neural networks. Em V. Feldman, A. Rakhlin & O. Shamir (Editores), *COLT*. (Ver página 60).
- Turing, A. M. (1950). Computing Machinery and Intelligence. *Mind*, 49, 433–460 (ver página 5).
- Weinberger, K. Q., Blitzer, J., & Lawrence, S. K. (2006). Distance Metric Learning for Large Margin Nearest Neighbor Classification. Em Y. Weiss, B. Schölkopf & J. C. Platt (Editores), *Advances in Neural Information Processing Systems 18* (páginas 1473–1480). MIT Press. (Ver página 22).
- Widrow, B., & Lehr, M. (1990). 30 years of adaptive neural networks: perceptron, Madaline, and backpropagation. *Proceedings of the IEEE*, 78(9), 1415–1442. <https://doi.org/10.1109/5.58323> (ver página 57).
- Xu, M., Quiroz, M., Kohn, R., & Sisson, S. A. (2019). Variance reduction properties of the reparameterization trick. *Artificial Intelligence and Statistics (AISTATS)* (ver página 42).